tf.compat.v1.autograph.to_graph Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/autograph/to_graph

> *Converts a Python entity into a TensorFlow graph.*

Function Signatures

```
tf.compat.v1.autograph.to_graph( entity, recursive=True,
arg_values=None, arg_types=None,
experimental_optional_features=None )
```

Code Examples

```
tf.compat.v1.autograph.to_graph(
entity,
recursive=True,
arg_values=None,
arg_types=None,
experimental_optional_features=None
)
```

```
tf.compat.v1.autograph.to_graph(
entity,
recursive=True,
arg_values=None,
arg_types=None,
experimental_optional_features=None
)
```

```
def foo(x):
if x > 0:
y = x * x
else:
y = -x
return y
converted_foo = to_graph(foo)
x = tf.constant(1)
y = converted_foo(x)  # converted_foo is a TensorFlow Op-like.
assert is_tensor(y)
```


```
def foo(x):
if x > 0:
y = x * x
else:
y = -x
return y
converted_foo = to_graph(foo)
x = tf.constant(1)
y = converted_foo(x)  # converted_foo is a TensorFlow Op-like.
assert is_tensor(y)
```


Documentation

Converts a Python entity into a TensorFlow graph.

Also see: tf.autograph.to_code, tf.function.

Unlike tf.function, to_graph is a low-level transpiler that converts
Python code to TensorFlow graph code. It does not implement any caching, variable management or create any actual ops, and is best used where greater control over the generated TensorFlow graph is desired. Another difference from tf.function is that to_graph will not wrap the graph into a
TensorFlow function or a Python callable. Internally, tf.function uses
to_graph.

Example Usage

Supported Python entities include:

- functions
- classes
- object methods

Functions are converted into new functions with converted code.

Classes are converted by generating a new class whose methods use converted code.

Methods are converted into unbound function that have an additional first argument called self.

Returns

Raises